# Unit 4 — Software Coding and Testing

This PDF explains every topic of Unit 4 in very simple language. It is made for final-exam revision. Read it from beginning to end, then revise only the last section before the exam.

## 1. Coding Standards

**Easy meaning:** Coding standards are fixed rules that programmers follow while writing code. They help everyone write code in the same style.

Coding standards make code clean, easy to read, easy to understand, and easy to repair. They also reduce mistakes.

### What Coding Standards Include

- How to name variables and functions.
- How to arrange and indent code.
- When to use global variables.
- How to write comments.
- How to handle errors.
- What information should be written at the top of a module.

### Examples of Coding Standards

- Use meaningful names: **studentMarks** is better than **x**.
- Use proper indentation so code looks organized.
- Write comments for difficult logic.
- Do not use too many global variables.
- Handle every error in the same way.

### Why Coding Standards Are Important

| Benefit | Simple Meaning |
|---|---|
| Consistency | All programmers write code in the same style. |
| Readability | Code is easy to read and understand. |
| Fewer errors | Common mistakes are avoided. |
| Easy maintenance | Bugs can be found and fixed easily. |
| Teamwork | Everyone can understand everyone else’s code. |
| Scalability | New features can be added easily. |

## 2. Coding Guidelines

**Easy meaning:** Coding guidelines are helpful suggestions for writing better programs. They are not always strict rules, but following them improves code quality.

### Important Guidelines

- Do not write confusing code.
- Do not use one variable for many different purposes.
- Write comments where needed.
- Keep functions small, preferably not more than 10 lines.
- Do not use the goto statement.
- Avoid deep nesting of if statements.
- Use meaningful names for variables and functions.
- Divide a big program into small functions or modules.

### Example

Bad variable name: **int x;**
Good variable name: **int studentMarks;**

The second name immediately tells us that the variable stores a student's marks.

**Exam answer:** Coding standards are rules and best practices followed to write clean, readable, consistent, and maintainable code. Coding guidelines are practical suggestions that help developers avoid mistakes and improve code quality.

## 3. Code Review

**Easy meaning:** Code review means checking another programmer's code before adding it to the main project. Senior developers or team members check whether the code is correct and follows rules.

### Purpose of Code Review

- To find mistakes early.
- To improve code quality.
- To check coding standards.
- To share knowledge among team members.
- To improve teamwork.
- To make sure the code works correctly.

### Code Review Workflow

1. Developer writes code.
2. Developer sends code for review.
3. Reviewer checks the code.
4. Reviewer gives comments.
5. Developer corrects the code.
6. Reviewer checks again.
7. Code is merged into the main project.

**Exam answer:** Code review is a systematic process in which developers examine another developer's code to find defects, improve quality, and ensure coding standards before the code is merged into the main codebase.

## 4. Code Walkthrough

**Easy meaning:** Code walkthrough is an informal method of checking code. Team members read the code and manually check what will happen when it runs.

### Process

1. The coder gives the code to team members.
2. Team members read and understand it.
3. They choose test cases.
4. They manually trace the code.
5. They discuss mistakes in a meeting.
6. The coder is present in the meeting.

### Main Purpose

A walkthrough mainly finds logical errors and algorithmic errors. For example, it can find a wrong formula or a missing condition.

**Exam answer:** Code walkthrough is an informal code-analysis technique in which team members manually examine code to discover algorithmic and logical errors.

## 5. Code Inspection

**Easy meaning:** Code inspection is a formal method of checking code. A team checks the code carefully for common programming mistakes and coding-standard violations.

### Inspection Team Roles

| Role | Work |
|---|---|
| Author | Writes the code. |
| Reader | Explains the code. |
| Inspector | Finds mistakes. |
| Recorder | Writes down defects. |

### Walkthrough vs Inspection

| Code Walkthrough | Code Inspection |
|---|---|
| Informal process. | Formal process. |
| Finds logical and algorithmic errors. | Finds common programming errors and standards violations. |
| Members manually trace code. | Code is checked systematically. |
| Less structured. | More structured. |

**Exam answer:** Code inspection is a formal review process in which a team examines code for common programming errors and checks whether coding standards are followed.

## 6. Common Programming Errors

- Using a variable before giving it a value.
- Creating an infinite loop.

- Using an invalid array index.
- Wrong use of logical operators such as AND and OR.
- Passing wrong arguments to a function.
- Changing a loop variable incorrectly.
- Improper memory allocation or deallocation.

### Example

If an array has 3 elements, valid indexes are 0, 1, and 2. Accessing index 3 is an array-index-out-of-bounds error.

## 7. Software Documentation

**Easy meaning:** Software documentation means written information about software. It explains what the software does, how to use it, how it was designed, and how to test it.

### Examples of Documents

- User manual.
- Design document.
- Test document.
- Installation manual.
- Software Requirements Specification, or SRS.

### Two Types

| Type | Where It Is Found | Example |
|---|---|---|
| Internal documentation | Inside the source code. | Comments and meaningful variable names. |
| External documentation | Outside the source code. | User manual, SRS, design document. |

**Exam answer:** Software documentation is the collection of documents prepared during software development, such as user manuals, SRS documents, design documents, and test documents.

## 8. Internal Documentation

**Easy meaning:** Internal documentation is written inside the program code. It helps programmers understand the code.

### Examples

- Comments.
- Module headers.
- Meaningful variable names.
- Function headers.
- Proper code structure.

Example: **// Calculate total marks of student** is internal documentation.

**Exam answer:** Internal documentation is documentation provided inside the source code through comments, module headers, meaningful names, and proper code structure.

# 9. External Documentation

**Easy meaning:** External documentation is written separately from the program. It helps users, testers, and customers understand the software.

## Examples

- User manual.
- Installation guide.
- SRS document.
- Design document.
- Test plan.

**Exam answer:** External documentation is documentation provided through separate supporting documents such as user manuals, SRS documents, design documents, and test documents.

# 10. Software Testing

**Easy meaning:** Software testing means checking software to find mistakes. We give inputs to the software and check whether the output is correct.

## Main Goals

- Find bugs and errors.
- Check whether software meets requirements.
- Find missing features.
- Check performance.
- Make software reliable before release.

## Example

If a login page accepts only correct username and password, testing checks whether wrong passwords are rejected and correct passwords are accepted.

**Exam answer:** Software testing is a systematic process of evaluating software to find defects and ensure that it meets specified requirements.

# 11. Verification and Validation

## Verification

**Easy meaning:** Verification checks whether the software is being built correctly according to rules, designs, and requirements.

It answers: **“Are we building the product right?”**

- It is usually a static process.

- Code may not be executed.
- It checks documents, design, and code.
- It is often done by the QA team.

### Validation

**Easy meaning:** Validation checks whether the finished software fulfils the customer's actual needs.

It answers: **"Are we building the right product?"**

- It is a dynamic process.
- Software is executed.
- It checks real behavior.
- It is done by testers and sometimes users.

### Simple Difference

| Verification | Validation |
|---|---|
| Building the product right. | Building the right product. |
| Static process. | Dynamic process. |
| No code execution needed. | Code execution is needed. |
| Checks documents, design, and code. | Checks actual software behavior. |
| Done by QA team. | Done by testing team or users. |

**Exam answer:** Verification ensures that software is developed according to specifications, while validation ensures that the final software meets customer requirements and real user needs.

## 12. Unit Testing

**Easy meaning:** Unit testing means testing the smallest part of a program separately. A unit can be a function, method, class, or small module.

### What Is Tested?

- Input and output of the module.
- Local variables and data structures.
- All possible paths in the code.
- Boundary values.
- Error-handling conditions.

### Example

If valid marks are from 0 to 100, test values such as -1, 0, 1, 99, 100, and 101.

**Exam answer:** Unit testing is the testing of the smallest testable part of software, such as a function, method, or class, to verify its internal logic and data handling.

## 13. Drivers and Stubs

**Easy meaning:** Sometimes we want to test one module, but the modules connected to it are not ready. Then we use temporary fake modules called drivers and stubs.

### Driver

**Easy meaning:** A driver is a fake main program that calls the module we want to test.

- Used in bottom-up testing.
- Simulates a higher-level module.
- It gives input to the module being tested.

### Stub

**Easy meaning:** A stub is a fake subprogram that replaces a lower-level module called by the module being tested.

- Used in top-down testing.
- Simulates a lower-level module.
- It returns a fixed or dummy response.

### Difference

| Driver | Stub |
|---|---|
| Dummy main program. | Dummy subprogram. |
| Calls the module being tested. | Is called by the module being tested. |
| Simulates higher-level module. | Simulates lower-level module. |
| Used in bottom-up testing. | Used in top-down testing. |

**Exam answer:** A driver is a dummy main program used to test lower-level modules, while a stub is a dummy subprogram used to replace lower-level modules during testing.

## 14. Integration Testing

**Easy meaning:** Integration testing checks whether two or more modules work properly together. It tests the connection or interface between modules.

### Errors Found

- Wrong data passing between modules.
- Wrong function calls.
- Interface mismatch.
- Communication problems.

### Three Approaches

| Approach | Simple Meaning | Temporary Component |
|---|---|---|
| Big Bang | Combine all modules at once and test. | Usually none. |

| | | |
|---|---|---|
| Bottom-up | Test low-level modules first. | Drivers. |
| Top-down | Test high-level modules first. | Stubs. |

**Exam answer:** Integration testing is the process of testing interfaces and interactions between two or more software modules to ensure they work together correctly.

# 15. Big Bang Integration Testing

**Easy meaning:** In Big Bang integration, all modules are combined together at one time and then tested as a complete system.

## Advantages

- Simple approach.
- No special integration order is needed.

## Disadvantages

- If an error occurs, it is difficult to find which module caused it.
- Debugging is difficult.

**Exam answer:** Big Bang integration testing combines all modules after individual testing and tests the complete integrated system at once.

# 16. Bottom-Up Integration Testing

**Easy meaning:** In bottom-up integration, testing starts from the lowest-level modules and moves upward.

## Process

1. Test small low-level modules first.
2. Combine them with higher modules.
3. Continue until the full system is tested.

- Drivers are used when higher-level modules are not ready.

## Advantage

- Basic modules are tested early.

## Disadvantage

- The full system behavior is tested late.

**Exam answer:** Bottom-up integration testing starts with the lowest-level modules and moves upward; drivers are used to simulate higher-level modules.

# 17. Top-Down Integration Testing

**Easy meaning:** In top-down integration, testing starts from the main or highest-level module and moves downward.

## Process

1. Test the main module first.
2. Add lower-level modules one by one.

3. Use stubs if lower modules are not ready.

### Advantage

- Main system behavior is tested early.

### Disadvantage

- Stubs must be created.

**Exam answer:** Top-down integration testing starts with the highest-level module and moves downward; stubs are used to simulate unavailable lower-level modules.

## 18. Regression Testing

**Easy meaning:** Regression testing means testing old features again after changing the software. It checks whether a new change has broken something that was working before.

### When Is It Done?

- After adding a new feature.
- After changing requirements.
- After fixing a bug.
- After improving performance.
- After changing the database or environment.

### Example

A website has email login. Facebook login is added. Regression testing checks whether email login still works after adding Facebook login.

**Exam answer:** Regression testing is repeated testing of an already tested program after modification to ensure that new changes have not introduced defects in previously working functionality.

## 19. Smoke Testing

**Easy meaning:** Smoke testing is a quick test done after a new build. It checks only the main and important features to see whether the build is stable.

### Example

For an online shopping website, smoke testing checks whether the site opens, login works, search works, and add-to-cart works.

### Smoke vs Regression

| Smoke Testing | Regression Testing |
|---|---|
| Quick test. | Detailed test. |
| Checks build stability. | Checks old features after changes. |
| Tests only critical functions. | Tests many existing functions. |
| Done after a new build. | Done after modification or bug fix. |

**Exam answer:** Smoke testing is a shallow and broad test performed after each build to verify that major and critical functionalities are working and the build is stable.

# 20. Validation Testing

**Easy meaning:** Validation testing checks whether the software fulfils the customer's business needs and requirements.

## It Checks

- Functional requirements.
- Behavioral requirements.
- Performance requirements.
- Informational requirements.

## Acceptance Testing

**Easy meaning:** Acceptance testing checks whether the customer should accept the software. It is done after system testing and before actual use.

**Exam answer:** Validation testing evaluates whether software satisfies specified business and customer requirements; acceptance testing determines whether the customer can accept the system.

# 21. Alpha Testing

**Easy meaning:** Alpha testing is done at the developer's office. Selected users use the software while developers watch them and note problems.

## Features

- Done at developer's site.
- Controlled environment.
- Developers are present.
- Users are representative end users.

**Exam answer:** Alpha testing is conducted at the developer's site by representative end users in a controlled environment, with developers observing and recording errors.

# 22. Beta Testing

**Easy meaning:** Beta testing is done at the user's place. Real users use the software in their normal environment and report problems to developers.

## Features

- Done at user sites.
- Real-world environment.
- Developers are usually absent.
- Users report problems.

## Alpha vs Beta

| Alpha Testing | Beta Testing |
|---|---|
| At developer’s site. | At user’s site. |
| Controlled environment. | Real environment. |
| Developers present. | Developers generally absent. |
| Before beta testing. | Before final release. |

**Exam answer:** Beta testing is conducted at end-user sites in a real environment where developers are generally absent; users report problems to developers.

# 23. System Testing

**Easy meaning:** System testing checks the complete software with all other parts of the system, such as hardware, database, network, and users.

## Types of System Testing

- Recovery testing.
- Security testing.
- Stress testing.
- Performance testing.
- Deployment testing.

**Exam answer:** System testing tests the complete integrated software along with hardware, database, network, and other system elements to verify that the whole system works correctly.

# 24. Recovery Testing

**Easy meaning:** Recovery testing checks whether software can recover after a failure.

## Examples

- Power failure.
- Network failure.
- Server crash.
- Database failure.

## MTTR

MTTR means Mean Time To Repair. It is the average time needed to repair the system after failure.

**Exam answer:** Recovery testing forces software failures and verifies whether the system can recover properly, either automatically or through human intervention.

# 25. Security Testing

**Easy meaning:** Security testing checks whether unauthorized people can access the software or data.

## Tester Acts Like a Hacker

- Tries to enter restricted pages.

- Tries wrong passwords.
- Tries to access another user's data.
- Tries malicious input.

**Exam answer:** Security testing verifies the software's protection mechanisms against improper penetration or unauthorized access.

# 26. Stress Testing

**Easy meaning:** Stress testing checks how software behaves when there is more load than normal.

## Examples

- Too many users at the same time.
- Too many requests.
- Very large files.
- Continuous data flow.

## Sensitivity Testing

Sensitivity testing is a type of stress testing. It checks how software behaves when one input value is changed beyond its normal limit.

**Exam answer:** Stress testing executes a system under abnormal resource demand, frequency, or volume to determine its failure point and recovery behavior.

# 27. Performance Testing

**Easy meaning:** Performance testing checks how fast and smoothly software works.

## It Measures

- Response time.
- Speed.
- Throughput.
- Resource usage.
- Scalability.

## Example

A performance test may check whether a search page gives results within 2 seconds.

**Exam answer:** Performance testing measures the runtime performance of software, including response time, throughput, resource usage, and scalability.

# 28. Deployment Testing

**Easy meaning:** Deployment testing checks whether software can be installed and used properly in the customer's environment.

## It Checks

- Installation steps.
- Configuration.
- Operating system compatibility.
- Database setup.
- Network setup.
- User documentation.

**Exam answer:** Deployment testing exercises software in each target environment and checks installation procedures, configuration, and user documentation.

# 29. White-Box Testing

**Easy meaning:** White-box testing means testing by looking inside the code. The tester knows the program logic, conditions, loops, and data structures.

## Other Names

- Glass-box testing.
- Structural testing.
- Clear-box testing.

## What Is Tested?

- All independent paths.
- All conditions, both true and false.
- Loops and their boundaries.
- Internal data structures.
- Every statement at least once.

## Basis Path Testing

**Easy meaning:** Basis path testing is a white-box technique that finds independent paths in a program and creates test cases for them.

## Example

For **if (age >= 18)**, test age = 18 for true condition and age = 17 for false condition.

**Exam answer:** White-box testing, also called glass-box or structural testing, examines the internal logic, control flow, and implementation of software to ensure all paths and conditions are tested.

# 30. Advantages and Disadvantages of White-Box Testing

## Advantages

- Tests all code paths and branches.
- Finds bugs early.
- Helps remove unnecessary code.
- Improves code quality.

- Can be automated.
- Helps find security weaknesses.

### Disadvantages

- Takes more time.
- Needs programming knowledge.
- Costly.
- Test cases change when code changes.
- May miss user-experience problems.

## 31. Prime Number Example

The following Java method checks whether a number is prime:

```
public static boolean isPrime(int num) {
  if (num <= 1) return false;
  for (int i = 2; i <= num / 2; i++) {
    if (num % i == 0) return false;
  }
  return true;
}
```

### Test Cases

| Input | Expected Output | Reason |
|---|---|---|
| 2 | true | 2 is prime. |
| 3 | true | 3 is prime. |
| 13 | true | 13 is prime. |
| 1 | false | 1 is not prime. |
| 0 | false | 0 is not prime. |
| -5 | false | Negative numbers are not prime. |

## 32. Black-Box Testing

**Easy meaning:** Black-box testing means testing software without looking inside the code. The tester only checks input and output.

### Other Names

- Functional testing.
- Behavioral testing.

### What Is Tested?

- Correct functions.
- Correct output.

- Invalid input handling.
- User interface.
- Performance.

### Errors Found

- Missing functions.
- Wrong functions.
- Interface errors.
- Database errors.
- Initialization and termination errors.

**Exam answer:** Black-box testing, also called functional or behavioral testing, tests software against functional requirements without considering its internal code structure.

## 33. Input Classes in Black-Box Testing

### Valid Input

Input that should be accepted. Example: age between 18 and 60.

### Invalid Input

Input that should be rejected. Example: negative age, empty field, or text instead of number.

### Boundary Values

Values at the edge of a valid range. If valid range is 1 to 100, test 0, 1, 2, 99, 100, and 101.

### Load and Stress

- Load testing: slowly increase users or data to find when the system slows down.
- Stress testing: push the system beyond its capacity.

**Exam answer:** Black-box testing uses valid inputs, invalid inputs, boundary values, and extreme data volumes to test software behavior and performance.

## 34. White-Box vs Black-Box Testing

| White-Box Testing | Black-Box Testing |
|---|---|
| Tests inside the code. | Tests outside behavior. |
| Needs source code. | Does not need source code. |
| Checks logic, loops, and paths. | Checks input, output, and functions. |
| Done by developers or technical testers. | Done by testers or users. |
| Finds internal coding errors. | Finds missing or incorrect functions. |
| Example: basis path testing. | Example: boundary value testing. |

**Exam answer:** White-box testing examines internal code structure, while black-box testing examines external software behavior based on requirements.

# Quick Revision — Read Before Exam

## Coding Standards and Guidelines

- Coding standards = rules for writing good code.
- They improve readability, consistency, and maintainability.
- Use meaningful names, comments, small functions, and proper indentation.
- Avoid goto, deep nesting, and multiple-purpose variables.

## Code Review

- Code review = checking code before merging.
- Walkthrough = informal; finds logic errors.
- Inspection = formal; finds common errors and standards violations.

## Documentation

- Internal documentation = inside code, such as comments.
- External documentation = separate documents, such as user manual and SRS.

## Verification and Validation

- Verification = Are we building the product right?
- Validation = Are we building the right product?
- Verification is static; validation is dynamic.

## Testing Levels

- Unit testing = smallest part.
- Integration testing = modules together.
- System testing = complete system.
- Validation testing = customer requirements.
- Acceptance testing = customer accepts or rejects the system.

## Drivers and Stubs

- Driver = dummy caller; bottom-up testing.
- Stub = dummy called module; top-down testing.

## Integration Approaches

- Big Bang = all modules at once.
- Bottom-up = low-level first; uses drivers.
- Top-down = high-level first; uses stubs.

## Important Testing Types

- Regression testing = retest after changes.
- Smoke testing = quick check of critical features after build.

- Alpha testing = developer site, controlled environment.
- Beta testing = user site, real environment.
- Recovery testing = recovery after failure.
- Security testing = protection from unauthorized access.
- Stress testing = beyond normal load.
- Performance testing = speed and resource usage.
- Deployment testing = installation and environment testing.

## White-Box and Black-Box

- White-box = internal code testing.
- Black-box = external behavior testing.
- Basis path testing = white-box technique.
- Boundary testing = black-box technique.

## Most Important Exam Differences

| Concept | One-Line Difference |
|---|---|
| Verification vs Validation | Verification checks process; validation checks customer need. |
| Walkthrough vs Inspection | Walkthrough is informal; inspection is formal. |
| Driver vs Stub | Driver calls module; stub is called by module. |
| Smoke vs Regression | Smoke checks build; regression checks old features after changes. |
| Alpha vs Beta | Alpha at developer site; beta at user site. |
| White-box vs Black-box | White-box tests code; black-box tests behavior. |